

Analiza filmów i platform streamingowych

Hanna Szczerbińska, Zuzanna Zalewska

Czerwiec 2026

1 Opis projektu

Głównym celem projektu było zaprojektowanie oraz implementacja architektury wielowarstwowej, która umożliwia użytkownikowi przeglądanie filmów pod kątem ich dostępności w wybranym regionie oraz na konkretnych platformach streamingowych. System pozwala także na szczegółową analizę rynku streamingowego poprzez generowanie różnych statystyk.

Link do repozytorium: https://github.com/zalewskaz/ASI_movies

2 Architektura systemu

System został zaprojektowany zgodnie z architekturą warstwową i składa się z następujących warstw i komponentów:

2.1 Źródła danych

System korzysta z dwóch zewnętrznych źródeł danych udostępniających informacje o filmach oraz platformach streamingowych: TMDb oraz Watchmode API. Są one dostępne pod adresami <https://developer.themoviedb.org/reference/intro/getting-started> oraz <https://api.watchmode.com/docs>. Oba źródła udostępniają dane poprzez interfejs REST API. Odpowiedzi zwracane są w formacie JSON, a dostęp do API realizowany jest przy użyciu API Key, przechowywanych lokalnie w pliku `.env`.

2.2 Moduł akwizycji danych

Data ingestion odpowiada za pobieranie danych z zewnętrznych źródeł API, ich przetwarzanie oraz synchronizację z lokalną bazą danych. Głównym celem modułu jest zapewnienie regularnej aktualizacji informacji o popularnych filmach oraz ich dostępności na platformach streamingowych.

Proces pobierania danych rozpoczyna się od komunikacji z API TMDb. System wykorzystuje endpoint `/movie/popular`, który zwraca listę kilkudziesięciu najpopularniejszych filmów. `tmdb_client.py` odpowiada za wykonywanie zapytań do API TMDb. Dla każdego pobranego filmu system odczytuje podstawowe informacje, takie jak TMDb id, tytuł filmu czy ścieżkę do plakatu. Następnie wykonywane jest mapowanie identyfikatora TMDb na identyfikator systemu Watchmode. Proces ten realizowany jest przez `watchmode_scraper.py`, który komunikuje się z API Watchmode przy użyciu endpointu wyszukiwania. Po uzyskaniu identyfikatora Watchmode system pobiera szczegółowe informacje dotyczące filmu, w szczególności dane o platformach streamingowych, regionach dostępności oraz cenach dostępu do filmu.

2.3 Moduł przetwarzania danych

Główna logika przetwarzania danych została zaimplementowana w pliku `data_pipeline.py`. Pipeline odpowiada za integrację danych z TMDb oraz Watchmode, a następnie ich walidację, przetwarzanie oraz synchronizację z bazą danych. Dla każdego filmu wykonywana jest weryfikacja dostępności danych w Watchmode. W przypadku braku aktywnych źródeł streamingowych rekord filmu oraz powiązane dane są usuwane z bazy

danych. Jeżeli dane są dostępne, system zapisuje informacje o filmie do tabeli `movies`. Następnie przetwarzane są dane dotyczące platform streamingowych. System grupuje oferty według pary (platforma, region), ponieważ dany film może posiadać wiele dostępnych opcji w ramach jednej platformy, np. różne plany (HD, 4K). W takim przypadku wybierany jest plan o najniższej dostępnej cenie.

System uwzględnia brakujące wartości w danych wejściowych. W przypadku braku informacji o ocenie użytkowników lub ocenie krytyków, pola te są uzupełniane wartością 0. Filmy zawierające takie wartości nie są uwzględniane w późniejszych statystykach dotyczących ocen, aby nie zniekształcać wyników. Analogicznie, w przypadku braku informacji o cenie dostępu do filmu, wartość ta ustawiana jest na 0 i również nie jest brana pod uwagę w statystykach cenowych. Jeżeli brak jest ścieżki do plakatu filmu, system przypisuje domyślną grafikę `placeholder.jpg`, zapewniając spójny wygląd interfejsu. Dodatkowo wartości ocen krytyków, które pierwotnie występują w zakresie 0–100, są normalizowane do skali 0–10 poprzez ich podzielenie przez 10, aby umożliwić bezpośrednie porównanie z ocenami użytkowników.

Dodatkowo pipeline działa w trybie cyklicznym jako osobny moduł odpowiedzialny za regularną synchronizację danych. Do realizacji harmonogramu wykorzystano bibliotekę `schedule`, a cały proces uruchamiany jest automatycznie raz w tygodniu.

2.4 Warstwa bazy danych

Warstwa bazy danych składa się z dwóch tabel: `movies` oraz `streaming`. Tabela `movies` przechowuje informacje o filmach, takie jak identyfikator TMDB (pełniący rolę klucza głównego), tytuł oraz ścieżka do plakatu. Dane te pochodzą z systemu TMDB. Pozostałe informacje, takie jak rok produkcji, oceny użytkowników i krytyków oraz czas trwania, są pobierane z systemu Watchmode, który dostarcza te dane w ramach zapytania o platformy streamingowe. Tabela `streaming` przechowuje informacje o dostępności filmów na platformach streamingowych. Każdy rekord zawiera sztuczny klucz główny, identyfikator filmu (klucz obcy do tabeli `movies`), nazwę platformy, region dostępności oraz cenę dostępu do filmu.

2.5 Backend

Warstwa backendowa odpowiada za udostępnianie interfejsu API, komunikację z bazą danych oraz przetwarzanie zapytań pochodzących z warstwy klienta. Zaimplementowano następujące endpointy:

- `/api/movies` – zwraca listę filmów z możliwością filtrowania po regionie oraz platformie streamingowej,
- `/api/filters/platforms` – zwraca listę unikalnych platform streamingowych,
- `/api/filters/regions` – zwraca listę unikalnych regionów,
- `/api/stats/platform-distribution` – zwraca rozkład liczby filmów dostępnych na poszczególnych platformach,
- `/api/stats/ratings/users` – zwraca średnie oceny użytkowników w podziale na platformy streamingowe,
- `/api/stats/ratings/critics` – zwraca średnie oceny krytyków w podziale na platformy streamingowe,
- `/api/stats/prices` – zwraca średnie ceny dostępu do filmów w zależności od regionu.

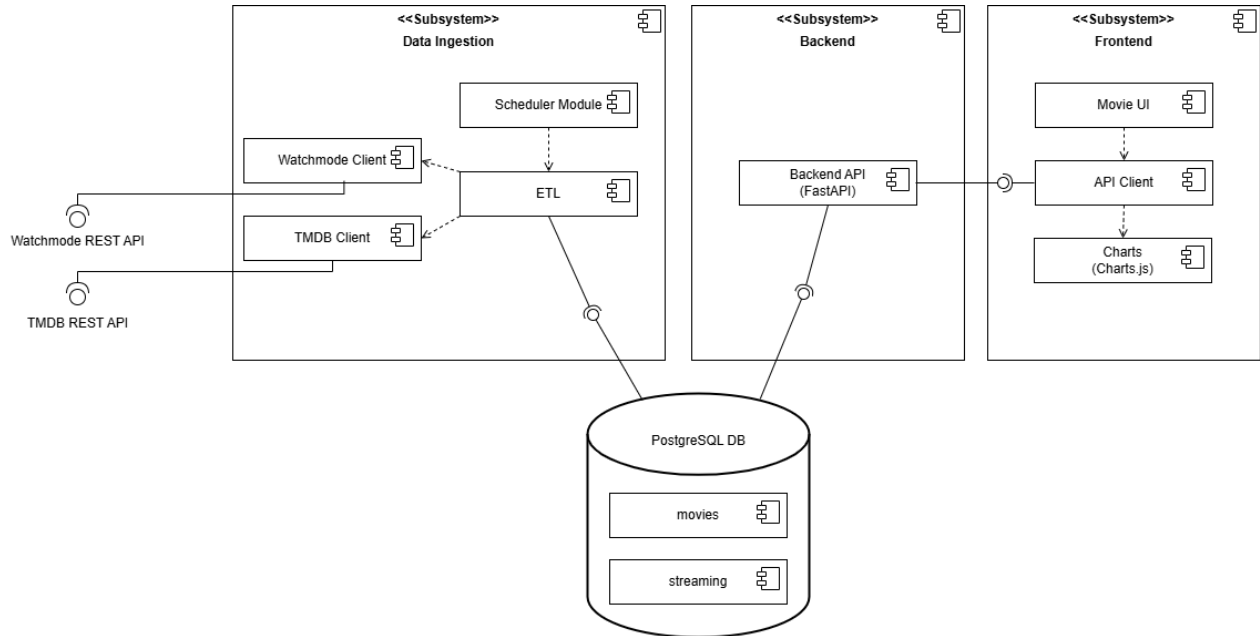
2.6 Frontend

Frontend odpowiada ona za prezentację danych użytkownikowi, obsługę interakcji oraz komunikację z warstwą backendową poprzez REST API. Interfejs użytkownika został podzielony na dwie zakładki: listę filmów oraz panel statystyk. Przełączanie pomiędzy widokami realizowane jest dynamicznie bez przeładowania strony. W widoku listy filmów użytkownik ma możliwość filtrowania danych według regionu oraz platform streamingowych. Wyniki są wyświetlane w formie kart filmów zawierających plakat, tytuł, rok produkcji, ocenę

użytkowników oraz czas trwania. Druga część interfejsu obejmuje moduł statystyk z wizualizacjami danych. W systemie zaimplementowano trzy typy wykresów: rozkład liczby filmów na platformach streamingowych, porównanie średnich ocen użytkowników i krytyków oraz średnia cena dostępu do filmów w zależności od regionu.

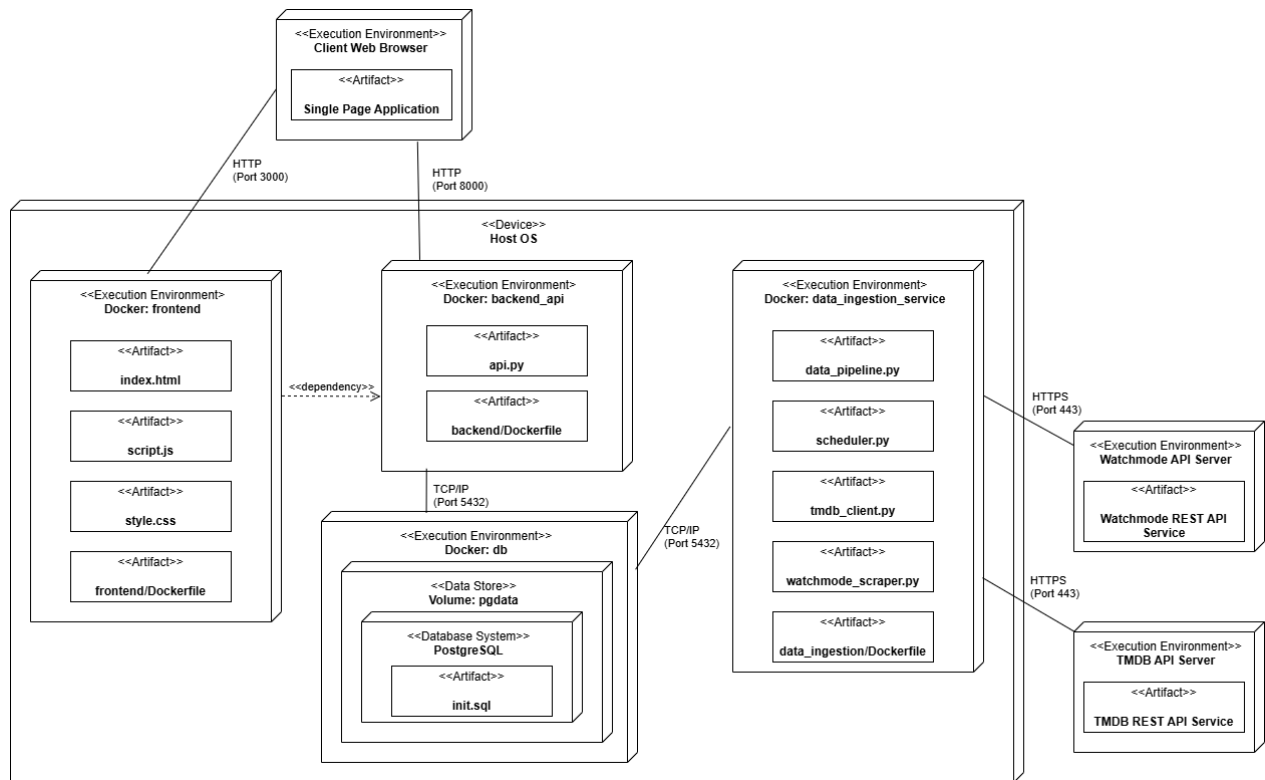
3 Diagramy architektury systemu

Logiczna struktura aplikacji oraz jej architektura wdrożeniowa zostały opisane przy użyciu dwóch diagramów UML. Na rysunku 1 przedstawiono *Component Diagram*. Obrazuje on podział odpowiedzialności pomiędzy moduły.



Rysunek 1: Diagram komponentów przedstawiający strukturę logiczną aplikacji

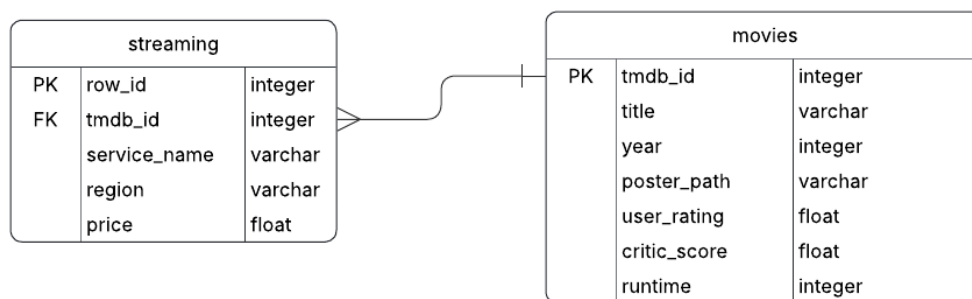
Na rysunku 2 przedstawiono *Deployment Diagram*, który odzwierciedla fizyczną strukturę systemu. Diagram wskazuje izolację procesów w ramach osobnych kontenerów (**Docker: frontend**, **Docker: backend_api**, **Docker: data_ingestion.service** oraz **Docker: db**), wolumen danych (**Volume: pgdata**), a także komunikację sieciową za pomocą protokołów TCP/IP oraz HTTP na dedykowanych portach.



Rysunek 2: Diagram wdrożenia systemu w środowisku kontenerowym Docker

4 Model danych

Na rysunku 3 przedstawiono diagram modelu bazy danych wykorzystanego w systemie. Zawiera on dwie tabele: **movies** oraz **streaming**, wraz z relacją jeden-do-wielu pomiędzy nimi, gdzie jeden film może posiadać wiele rekordów dotyczących dostępności na platformach streamingowych.



Rysunek 3: Diagram ER przedstawiający strukturę bazy danych systemu

5 Zastosowane technologie

W tabeli 1 przedstawiono szczegółowy opis zastosowanych technologii wraz z uzasadnieniem ich wyboru.

Technologia	Zastosowanie	Uzasadnienie wyboru
Python	Logika backendowa i pipeline przetwarzania danych	Wszechstronność, bogaty zbiór bibliotek do analizy danych i tworzenia API, szybkość implementacji, znajomość języka
FastAPI	Implementacja REST API	Bardzo wysoka wydajność dzięki asynchroniczności; zautomatyzowana generacja dokumentacji (Swagger)
PostgreSQL	Relacyjna baza danych	Niezawodność, wysoka wydajność przy relacyjnych modelach danych, dobra integracja z Dockerem
psycopg2	Komunikacja z bazą danych	Interfejs komunikacji między Pythonem a bazą danych, umożliwiający stosowanie zapytań parametryzowanych
schedule	Harmonogram zadań	Biblioteka umożliwiająca cykliczne uruchamianie zadań w tle bez konieczności konfiguracji menedżera zadań na poziomie OS.
HTML / CSS	Interfejs użytkownika	Standardowe, proste technologie webowe umożliwiające stworzenie UI
JavaScript	Logika frontendowa	Asynchroniczna obsługa interfejsu po stronie klienta pozwalająca na płynne przełączanie zakładek i filtrowanie bez ponownego ładowania strony internetowej
Chart.js i Font Awesome	Wizualizacja danych	Generowanie przejrzystych, interaktywnych wykresów w czasie rzeczywistym
Docker	Konteneryzacja aplikacji i środowiska uruchomieniowego	Izolacja usług, łatwość przenoszenia projektu pomiędzy urządzeniami, symulacja warunków produkcyjnych
Pytest	Testy jednostkowe	Proste i czytelne narzędzie do automatyzacji testów kodu Python, ułatwiające weryfikację logiki biznesowej i poprawności transformacji danych
Locust	Testy wydajnościowe i obciążeniowe	Narzędzie umożliwiające łatwe definiowanie scenariuszy użytkowników i generowanie masowego, równoległego ruchu

Tabela 1: Technologie zastosowane w projekcie

6 Dokumentacja API

Framework FastAPI automatycznie generuje dokumentację API w Swagger UI (<http://localhost:8000/docs#/>). Umożliwia ona przede wszystkim przegląd wszystkich zaimplementowanych przez nas endpointów. Na rysunku 4 przedstawiono widok dokumentacji endpointów REST API wygenerowanej przez Swagger.

ASI Movies API ^{1.0} ^{OAS 3.1}

/openapi.json

default ^		
GET	/api/movies	Get Movies
GET	/api/filters/platforms	Get Unique Platforms
GET	/api/filters/regions	Get Unique Regions
GET	/api/stats/platform-distribution	Get Platform Distribution
GET	/api/stats/ratings/users	Get User Ratings
GET	/api/stats/ratings/critics	Get Critic Ratings
GET	/api/stats/prices	Get Prices

Rysunek 4: Dokumentacja endpointów REST API wygenerowana przez Swagger

7 Testy

7.1 Testy jednostkowe

Testy jednostkowe zostały zaimplementowane za pomocą frameworku `pytest`; mają one na celu weryfikację poprawności backendowego API oraz modułu akwizycji danych. W celu testowania komponentów w sposób możliwie niezależny od innych elementów infrastruktury (np. działania API czy lokalnej bazy danych), wykorzystano mechanizm mockowania z biblioteki `unittest.mock`.

Dla warstwy REST API przetestowano wszystkie utworzone endpointy, symulując zapytania SQL oraz struktury danych zwracane przez bazę. Zweryfikowano również działanie API w przypadkach brzegowych (np. gdy nie zostały przekazane wymagane parametry).

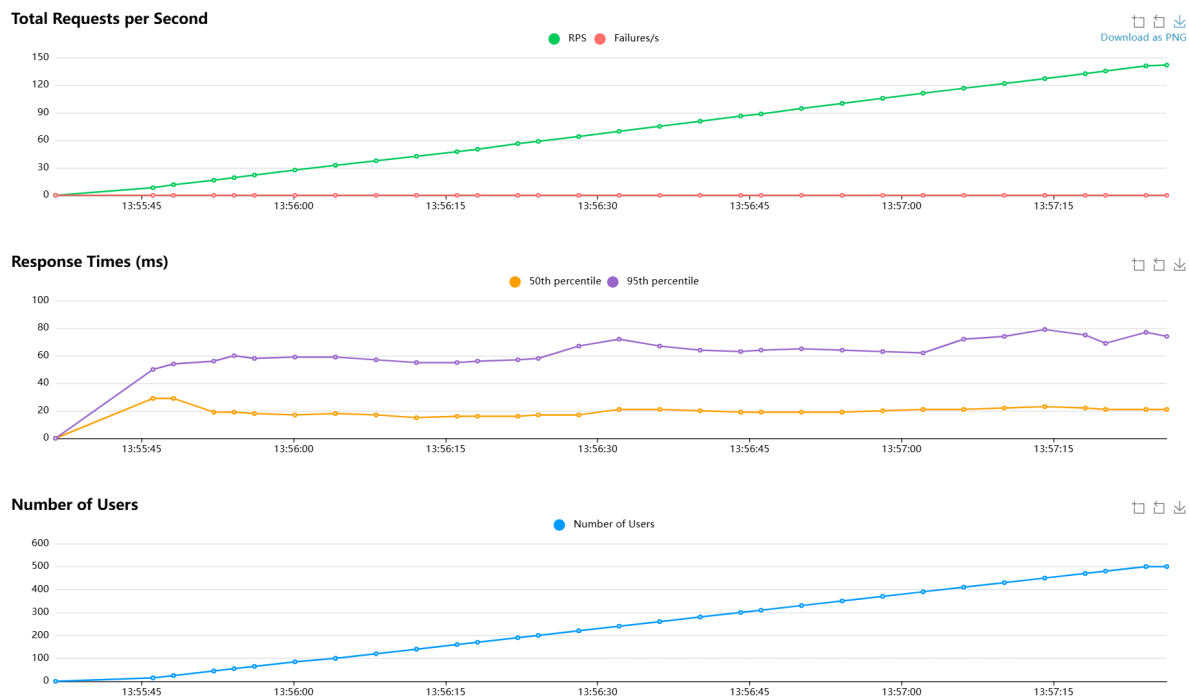
W przypadku modułu pobierającego dane, zasymulowano odpowiedzi z systemów TMDb i Watchmode w formacie JSON, i za ich pomocą przetestowano logikę przekształcania danych (w tym obsługi błędów i brakujących kluczy).

7.2 Testy wydajnościowe

Testy wydajnościowe i obciążeniowe zostały zrealizowane przy pomocy narzędzia `Locust` (skrypty scenariusza testowego umieszczono w katalogu `tests/load/`). Testy te są uruchamiane wewnątrz wydzielonego środowiska kontenerowego, w którym dedykowany generator ruchu symuluje masowe, równoległe żądania użytkowników skierowane do testowej instancji API. W ramach scenariusza zaimplementowano pięć konkretnych zadań odzwierciedlających zachowania użytkowników na stronie:

- Ładowanie strony głównej – żądanie pobierające pełną listę filmów bez nałożonych filtrów.
- Filtrowanie filmów – odpytywanie bazy danych z losową kombinacją parametrów regionu oraz wybranych platform streamingowych.
- Przegląd statystyk platform – pobieranie danych o rozkładzie liczby filmów dostępnych w poszczególnych serwisach.
- Analiza statystyk cenowych – sprawdzanie średnich kosztów dostępu do filmów w losowo wybranym regionie geograficznym.
- Porównanie ocen – pobranie średnich ocen wystawionych zarówno przez użytkowników, jak i krytyków filmowych.

Interfejs graficzny narzędzia testowego jest dostępny pod adresem <http://localhost:8089>. Wyniki tego testu są automatycznie wygenerowane w panelu Locust w postaci trzech wykresów wydajnościowych: liczby żądań na sekundę, czasów odpowiedzi systemu oraz aktualnej liczby symulowanych użytkowników. Na rysunku 5 przedstawiono przykładowe testowanie systemu pod obciążeniem 500 równoległych użytkowników, przyrastających w tempie 5 użytkowników na sekundę.



Rysunek 5: Wykresy wydajnościowe z panelu Locust dla 500 użytkowników (Ramp-up: 5)

8 Środowiska uruchomieniowe

Architektura wdrożeniowa została oparta na konteneryzacji, z podziałem na trzy wyizolowane środowiska uruchomieniowe zarządzane za pomocą dedykowanych plików konfiguracji docker-compose.

- Środowisko deweloperskie (DEV): Przeznaczone do bieżącej pracy deweloperskiej podczas tworzenia nowych funkcjonalności. W tym środowisku zastosowano mechanizm wolumenów, które mapują lokalne pliki bezpośrednio do pamięci kontenerów. Dzięki temu zmiany w kodzie są natychmiastowo odzwierciedlane w aplikacji bez konieczności każdorazowego przebudowywania obrazów Dockera (tzw. hot-reloading za pomocą flagi `--reload` we frameworku FastAPI oraz w plikach JavaScript).

- Środowisko testowe (TEST): Przeznaczone do zautomatyzowanego testowania kodu przed jego implementacją w środowisku produkcyjnym. W środowisku testowym jest uruchamiana dedykowana, oddzielna instancja bazy danych, dzięki czemu skrypty testowe nie mają wpływu na główną bazę danych, od której zależy również środowisko produkcyjne. W tym środowisku zostały również zaimplementowane testy jednostkowe (za pomocą biblioteki `pytest`) oraz testy obciążeniowe (za pomocą narzędzia `Locust`).
- Środowisko produkcyjne (PROD): Przeznaczone do uruchamiania ostatecznej aplikacji, która może następnie zostać udostępniona użytkownikom. To środowisko jest zoptymalizowane pod kątem wydajności i stabilności: kod źródłowy jest statycznie wbudowany i skompilowany z obrazem Dockera, serwer backendowy uruchamiany jest w sposób ciągły bez mechanizmów nasłuchujących na zmiany plików, a baza danych jest odizolowana od zewnętrznej sieci.

9 Podział pracy

W tabeli 2 przedstawiono macierz odpowiedzialności, obrazującą szczegółowy podział funkcjonalności oraz zadań zrealizowanych przez poszczególne członkinie zespołu projektowego.

Zadanie	H. Szczerbińska	Z. Zalewska
Integracja zewnętrznych API	✓	✓
Implementacja logiki przepływu i transformacji danych	✓	✓
Konfiguracja harmonogramu pobierania danych (<code>schedule</code>)	✓	
Projekt i implementacja bazy danych PostgreSQL	✓	
Stworzenie endpointów (FastAPI)		✓
Budowa warstwy wizualnej frontendu	✓	✓
Konteneryzacja komponentów systemu	✓	✓
Konfiguracja środowiska produkcyjnego i deweloperskiego	✓	
Konfiguracja środowiska testowego		✓
Implementacja i uruchomienie testów obciążeniowych (Locust)		✓
Implementacja testów jednostkowych (pytest)	✓	
Przygotowanie diagramów UML		✓
Stworzenie dokumentacji	✓	✓

Tabela 2: Podział prac w zespole